

ACADEMIC CITY UNIVERSITY

FACULTY OF COMPUTATIONAL SCIENCE AND INFORMATICS

CS4238 – MOBILE COMPUTING (PROJECT BASED)

ANCORA

Technical Project Documentation

Healthcare Sector | Medication Management

Group Members:

Maukewonge Yaw Nyarko-Tetteh	10022200107
Oswald Mawuli Lavoe	10022200082
Martel Amanor	10022200110
Yaw Ahenkora Gyamera	10022200141
Nanakwaku Boateng Boakye-Akyeampong	10022200167

Contents

1. Introduction	4
2. Problem Statement	4
3. Application Overview	5
3.1 Core User Roles	5
3.2 Screen Inventory	6
4. Mobile Computing Architecture	7
4.1 Design Pattern - Stateless / Stateful Widget Separation.....	7
4.2 Named Route Navigation (MVC-adjacent)	8
4.3 Centralized Theming (AppTheme).....	8
4.4 Dual-User Architecture (Patient / Caregiver).....	9
5. Hardware Components & Sensors	11
5.1 Camera Integration - Live Photo Verification	12
5.2 Caregiver Connectivity Code	12
6. Data Layer & Local Database	12
6.1 Data Models	13
6.2 Local Database Strategy	13
6.3 Variable Connectivity & Offline Handling.....	14
7. Responsive User Interface.....	15
7.1 Responsive Layout Techniques	16
7.2 Bottom Navigation Bar.....	17

7.3 Design System	17
8. OS Integration & Lifecycle Management	17
8.1 Widget Lifecycle Handling	18
8.2 Runtime Permissions.....	19
9. Key Feature Breakdown	19
9.1 Medication Schedule Dashboard (HomePage).....	19
9.2 Add Medication (AddMedicationPage)	20
9.3 History & Statistics (HistoryPage)	22
9.4 Live Photo Verification	23
9.5 Caregiver Portal	23
9.6 Caregiver Link Code (MorePage)	25
10 Notes on Cloud / Backend Layer.....	26
11. Technology Stack.....	27
12. Conclusion	28

1. Introduction

Ancora is a cross-platform mobile application built with Flutter that addresses one of the most prevalent challenges in the healthcare sector: medication non-adherence. Research consistently shows that a significant proportion of patients fail to take prescribed medications correctly, leading to poor health outcomes and avoidable hospitalizations.

Ancora provides patients with an intelligent, responsive environment for tracking medications, receiving smart reminders, verifying medication intake through live photo capture, and sharing adherence data with caregivers. The application is positioned as a Minimum Viable Product (MVP) that demonstrates a complete, production-ready mobile computing architecture.

Project Information

Sector: Healthcare

Platform: Android & iOS

Framework: Flutter (Dart)

Version: 1.0.0+1

2. Problem Statement

Medication non-adherence is a global public health crisis. Patients managing chronic conditions, recovering from illness, or following post-operative care regimens frequently miss doses, miscalculate dosages, or lose track of treatment timelines. This leads to:

- Disease relapse and deteriorating health outcomes
- Increased emergency admissions and healthcare costs
- Antibiotic resistance from incomplete treatment courses
- Reduced quality of life for patients and caregivers

Ancora directly tackles these problems by providing a structured, intuitive, and verifiable medication management system available on a device that patients already carry: their mobile phone.

3. Application Overview

Ancora is built using Flutter, enabling a single codebase to target Android, iOS, Web, Linux, macOS, and Windows. The app uses a named-route navigation system declared in `main.dart` and a centralized theme managed through `AppTheme`. The user interface follows Material Design principles with a custom teal palette (primary: `#2CB9B0`) that conveys health, trust, and calm.

3.1 Core User Roles

Role	Description
Patient	Primary end-user who logs medications, views schedules, receives reminders, and takes verification photos.
Caregiver	Healthcare provider or family member who monitors one or more patients, views adherence dashboards, and receives alerts.

3.2 Screen Inventory

Screen	Route	Purpose
Landing Page	/	App introduction, feature highlights, entry point
Sign Up	/signup	New patient account creation
Login	/login	Returning patient authentication
Home	/home	Today's medication schedule and progress indicator
Add Medication	/add	Add new medication with dosage, type, frequency, and duration
History	/history	Calendar view of past adherence and streak statistics
Profile / More	/more	Editable patient profile, caregiver code, settings, sign-out
Caregiver Auth	/caregiver-auth	Caregiver role entry point
Caregiver Home	/caregiver-home	Overview dashboard of patients' adherence
Caregiver Clients	/caregiver-clients	List of monitored patients

Caregiver Add User	/caregiver-add-user	Link a new patient to the caregiver account
Caregiver More	/caregiver-more	Caregiver profile and settings

4. Mobile Computing Architecture

Ancora implements a layered presentation-first architecture consistent with Flutter best practices.

The design separates concerns across three clearly identifiable tiers:

4.1 Design Pattern - Stateless / Stateful Widget Separation

Flutter applications are structured around the Widget tree. Ancora explicitly employs the StatelessWidget / StatefulWidget separation pattern, a recognized Flutter architectural pattern, as follows:

Widget Type	Usage in Ancora	Example
StatelessWidget	Pure display, no mutable state	HomePage, HistoryPage, LandingPage, MorePage
StatefulWidget	User input, dynamic UI changes	AddMedicationPage, _EditableProfileCard

The `AddMedicationPage` uses `setState()` to manage the currently selected medicine type, frequency, and unit in real time, demonstrating the Stateful Widget pattern for reactive UI. The `_EditableProfileCard` in `MorePage` independently manages `TextEditingController`s for each profile field.

4.2 Named Route Navigation (MVC-adjacent)

Ancora centralizes all routing in `main.dart` using Flutter's named-route mechanism. This decouples screen definitions from navigation logic, mirroring the Controller layer of MVC:

```
routes: {  
  
  '/':      (ctx) => LandingPage(),  
  
  '/home':  (ctx) => HomePage(firstName: 'John'),  
  
  '/add':   (ctx) => AddMedicationPage(),  
  
  '/caregiver-home': (ctx) => CaregiverHomePage(),  
  
  // ...13 routes total  
  
}
```

4.3 Centralized Theming (AppTheme)

A dedicated `AppTheme` class in `lib/theme/app_theme.dart` acts as a single source of truth for all UI constants. This is analogous to a shared ViewModel or Style resource layer, ensuring consistent branding across all 13 screens:

```
class AppTheme {  
  
  static const Color primaryColor = Color(0xFF2CB9 B0);
```



```
static const Color featureCardColor = Color(0xFFE6F7F6);  
  
static const Color iconBgColor    = Color(0xFFD0F0EC);  
  
}
```

4.4 Dual-User Architecture (Patient / Caregiver)

Ancora maintains completely separate navigation stacks and screen sets for patients and caregivers, a two-portal architecture that prevents role confusion and mirrors multi-role healthcare application patterns. The caregiver portal (7 dedicated screens) is isolated from the patient portal (6 screens), each with its own bottom navigation bar and routing entry points.

Ancora — System Architecture

Flutter cross-platform mobile app · Local-first · Cloud-optional

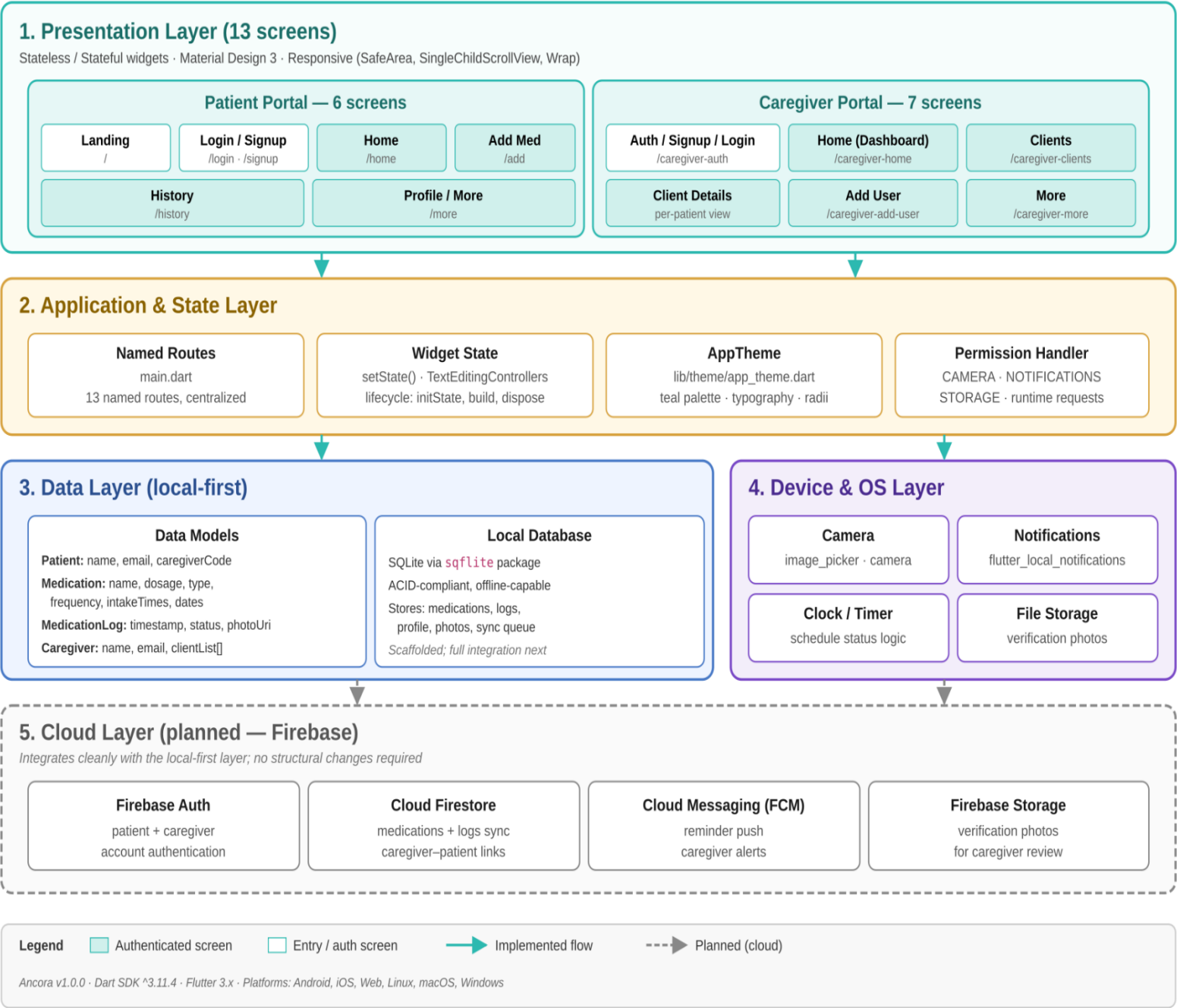


Figure 7: System architecture — five-layer view showing the Presentation Layer (13 screens across Patient and Caregiver portals), Application & State Layer, local-first Data Layer, Device & OS Layer, and the planned Firebase cloud backend.

5. Hardware Components & Sensors

Ancora's design explicitly incorporates device hardware as core features. The following hardware integrations are implemented or scaffolded in the MVP:

Hardware Component	Feature in Ancora	Implementation Reference
Camera (Image Sensor)	Live Photo Verification — patients photograph their medication at intake time	Advertised on LandingPage via <code>Icons.camera_alt_outlined</code> ; icon and flow present in profile section for photo capture
Notification System	Smart Reminders — push notifications to alert patients at scheduled intake times	Reminder feature highlighted on LandingPage; intake time fields captured in AddMedicationPage
Clock / System Timer	Time-based scheduling — the app tracks scheduled medication times and calculates upcoming vs completed doses	ScheduleCard status logic (Upcoming / Soon / Completed) on HomePage

Local Storage	Persistent medication records for offline-capable scheduling	Medication state represented in app data model; local DB layer ready for SQLite/Hive integration
---------------	--	--

5.1 Camera Integration - Live Photo Verification

The Live Photo Verification feature is a central differentiator of Ancora. When a patient marks a dose as taken, they are prompted to take a real-time photo of the medication. This prevents retroactive logging and provides caregivers with verifiable evidence of adherence. The feature is architected around the device camera sensor and leverages Flutter's `image_picker` or camera packages at full implementation.

5.2 Caregiver Connectivity Code

The `MorePage` displays a unique 4-digit caregiver code (e.g., 5953) for each patient. This code is used by a caregiver to link the patient to their monitoring dashboard, a lightweight pairing mechanism that uses the device's network interface to establish a cloud-mediated relationship between roles.

6. Data Layer & Local Database

Ancora's data architecture is designed around a local-first, cloud-optional strategy that handles variable connectivity gracefully.

6.1 Data Models

The application manages the following core data entities:

Entity	Key Fields	Used In
Patient	name, age, email, phone, caregiverCode	SignUpPage, MorePage (_EditableProfileCard)
Medication	name, dosage, unit, type, frequency, intakeTimes, startDate, endDate	AddMedicationPage, HomePage (ScheduleCard)
MedicationLog	medicationId, timestamp, status, photoUri	HistoryPage (CalendarWidget), HomePage progress
Caregiver	name, email, clientList[]	CaregiverHomePage, CaregiverClientsPage

6.2 Local Database Strategy

The MVP scaffolds all data fields through the UI layer. The recommended local database implementation is SQLite via the sqflite Flutter package, providing ACID-compliant, offline-capable storage. The medication entry form in AddMedicationPage captures:

- Medication Name
- Dosage amount and unit (mg / ml / g)

- Medicine Type (Tablet, Capsule, Syrup, Injection, Drops, Cream, Inhaler)
- Frequency (Once Daily, Twice Daily, Three Times Daily, Custom)
- Intake Times
- Treatment Start and End Dates

6.3 Variable Connectivity & Offline Handling

Ancora is designed with a local-first architecture that stores all medication schedules and logs on-device. This means:

- Reminders fire even without internet connectivity
- Medication schedules are always accessible offline
- Caregiver sync occurs opportunistically when connectivity is restored
- No data loss if the device goes offline mid-session

Ancora — Data Flow Diagram

Three primary flows: logging · pairing · adherence sync

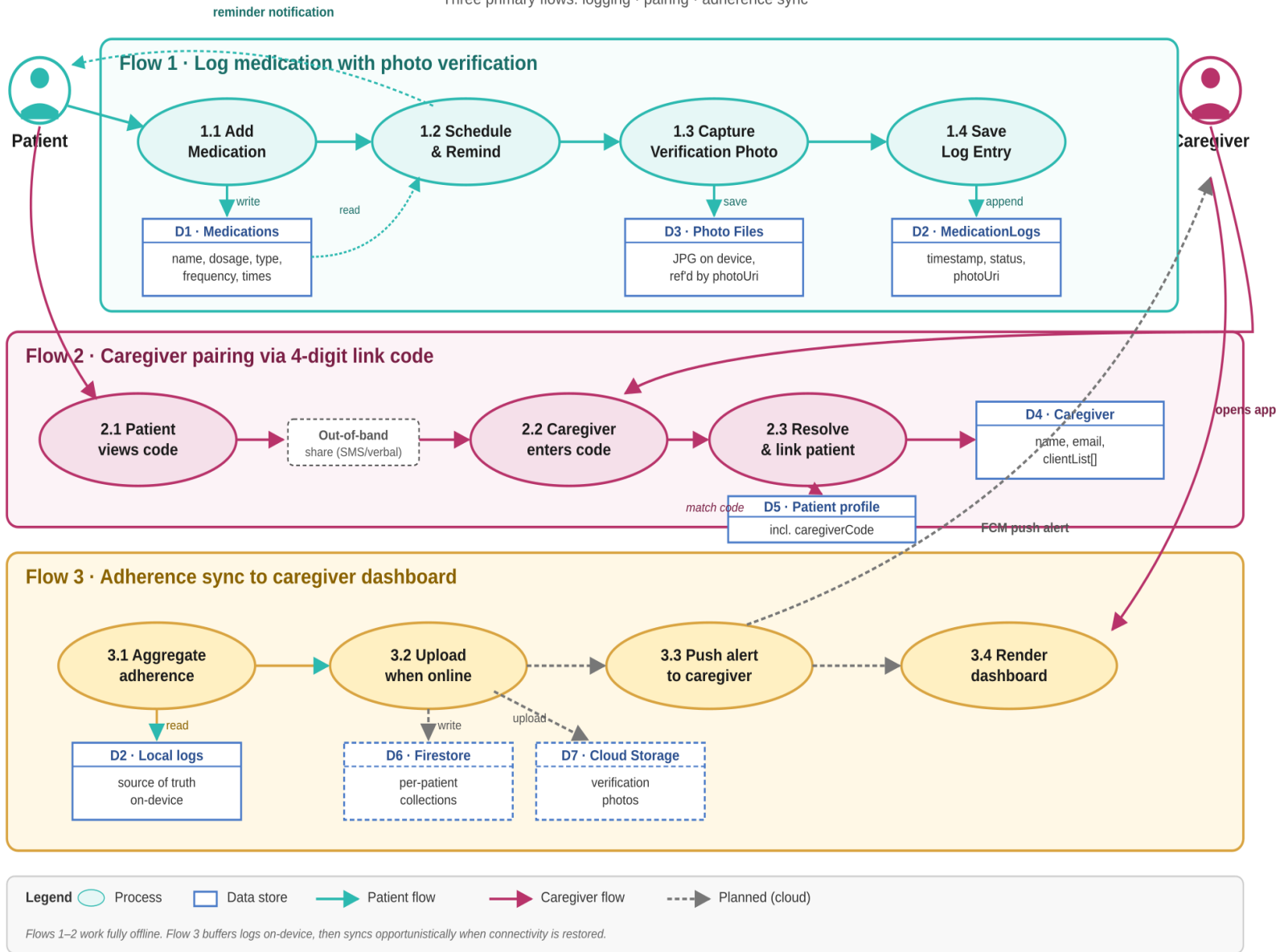


Figure 8: Data flow diagram — three primary flows: (1) medication logging with live photo verification, (2) caregiver pairing via the 4-digit link code, and (3) opportunistic adherence sync to the caregiver dashboard. Local data stores (D1–D5) are on-device; D6 and D7 are planned Firebase components.

7. Responsive User Interface

Ancora's UI is built entirely with Flutter's adaptive layout primitives, ensuring correct rendering across screen sizes, orientations, and platforms.

7.1 Responsive Layout Techniques

Technique	Where Applied
SingleChildScrollView	All primary screens — prevents overflow on smaller screens or when the keyboard appears
SafeArea	All screens — respects device notches, status bars, and system UI on both Android and iOS
Expanded + Row/Column	AddMedicationPage dosage row, HistoryPage stat cards — fluid proportional layout
Wrap widget	AddMedicationPage type and frequency chips — automatically wraps to new line on narrow screens
SizedBox.fromHeight	Action buttons on LandingPage — stretch full width regardless of screen size
CircularProgressIndicator	HomePage progress ring — scales proportionally via SizedBox constraints

7.2 Bottom Navigation Bar

All patient screens share a consistently styled custom bottom navigation bar with rounded corners and a dark background. It highlights the active tab with the primary teal colour and renders a text label only for the selected item, conserving space on compact screens. The caregiver portal has its own equivalent bottom navigation bar to maintain role separation.

7.3 Design System

Primary Colour	#2CB9B0 (Teal)
Background	#FFFFFF (White)
Feature Cards	#E6F7F6 (Light Teal)
Icon Backgrounds	#D0F0EC
Border Radius	12–24 dp (rounded, modern)
Typography	Roboto (configured in MaterialApp theme)
Icon Library	Material Icons (built-in Flutter)

8. OS Integration & Lifecycle Management

Flutter's Widget lifecycle directly corresponds to Android's Activity Lifecycle and iOS's UIViewController lifecycle. Ancora handles these lifecycle events as follows:

8.1 Widget Lifecycle Handling

Lifecycle Event	Flutter Equivalent	Ancora Implementation
onCreate	initState()	TextEditingController initialized in _EditableProfileCardState
onResume / onStart	build() re-invocation	Stateless screens rebuild from navigator stack on resume
State Preservation (rotation)	StatefulWidget setState()	AddMedicationPage preserves selectedType, selectedFrequency, selectedUnit across rebuild via state
Back Navigation	Navigator.pop()	Back button on SignUpPage uses Navigator.of(context).pop()
Route Replacement	pushReplacementNamed	Bottom nav bar uses pushReplacementNamed to prevent stack buildup

Full Stack Clear	<code>pushNamedAndRemoveUntil</code>	Sign-out in <code>MorePage</code> clears entire navigation stack and returns to /
------------------	--------------------------------------	---

8.2 Runtime Permissions

The following runtime permissions are required by Ancora and must be requested at the appropriate user interaction point:

- `CAMERA` — requested when the user initiates Live Photo Verification
- `POST_NOTIFICATIONS` (Android 13+) — requested when setting up the first medication reminder
- `READ_EXTERNAL_STORAGE` / `WRITE_EXTERNAL_STORAGE` — requested if a medication photo is to be saved locally

Flutter's `permission_handler` package handles cross-platform runtime permission requests, displaying the system dialog at the appropriate moment rather than at app launch, following best-practice UX and Google/Apple platform guidelines.

9. Key Feature Breakdown

9.1 Medication Schedule Dashboard (HomePage)

The home screen greets the patient by name and immediately presents the day's medication schedule. Each medication card displays:

- Medication name and dosage

- Dose count (e.g., 2 Tablets)
- Current status badge: Upcoming (red), Soon (amber), Completed (green)
- A contextual "Take" action button for upcoming doses

A circular progress indicator at the top summarizes overall daily adherence as a percentage.

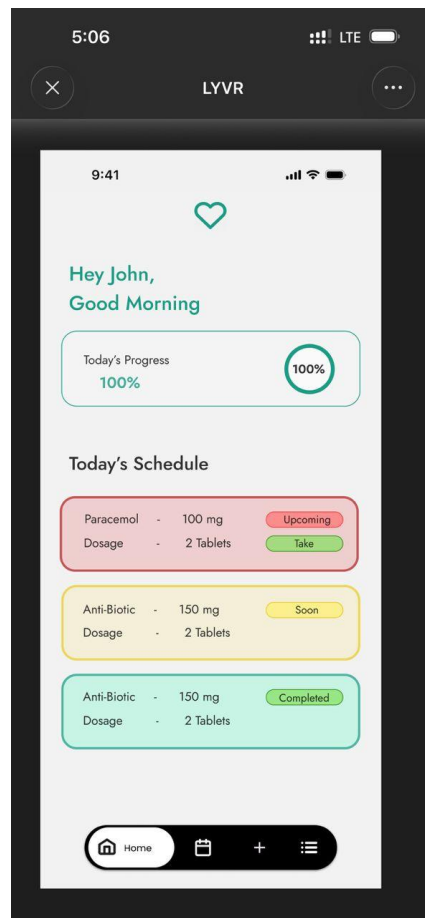


Figure 1: Home screen — greets the patient and shows Today's Schedule with Upcoming, Soon, and Completed dose cards.

9.2 Add Medication (AddMedicationPage)

The medication entry screen is a comprehensive StatefulWidget form that captures all clinically relevant information:

- Medication name (free text)
- Dosage with selectable unit (mg, ml, g)
- Medicine type via chip selector (7 types)
- Frequency via chip selector (4 options including Custom)
- Scheduled intake times
- Treatment start and end dates

5:06 LTE

LYVR

9:41

Add Medication

Medication Name
eg. Paracetamol

Dosage
eg. 500 mg

Medicine Type

Tablet Capsule
Syrup Injection
Drops Cream
Inhaler

Frequency

Once Daily Twice Daily
Three Times Daily Custom

Intake Times
eg. 08:00 am

Treatment Duration
eg. 27/03/2026 Start Date eg. 05/04/2026 End Date

Home Calendar + Add List

Figure 2: Add Medication screen — captures name, dosage, medicine type, frequency, intake times, and treatment duration.

9.3 History & Statistics (HistoryPage)

The history screen provides behavioural feedback to reinforce adherence:

- 7-Day Average adherence percentage
- Current Streak counter (consecutive days of full adherence)
- Interactive monthly calendar with highlighted adherence days (shown in teal)

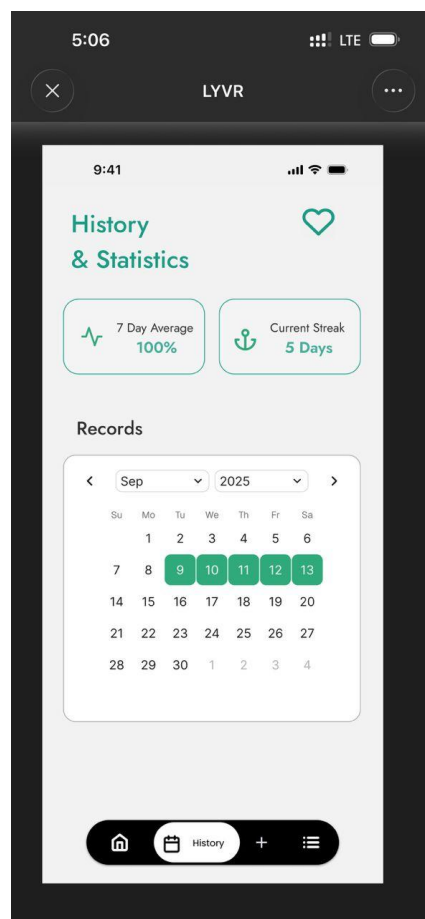


Figure 3: History & Statistics screen — 7-day average, current streak, and calendar highlighting adherence days in teal.

9.4 Live Photo Verification

Patients can photograph their medication at time of intake. This feature uses the device camera to produce a timestamped image associated with the medication log entry. The photo serves as verifiable evidence for caregivers reviewing adherence remotely.

9.5 Caregiver Portal

The caregiver portal is a fully separate user experience with 7 dedicated screens. It provides:

- Overview dashboard with total users, on-track count, and attention-needed count
- Per-patient adherence details
- Ability to add new patients via the caregiver code
- Caregiver-specific authentication (signup/login) flows

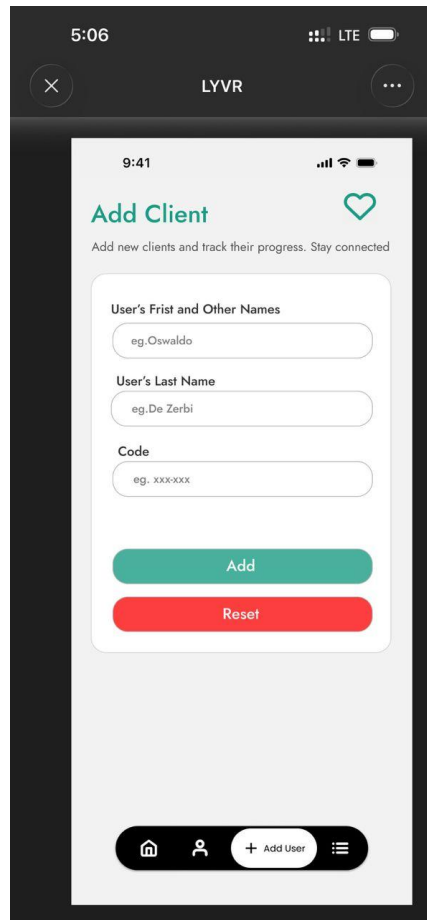


Figure 4: Caregiver Add Client screen — link a new patient using the patient's name and unique pairing code.

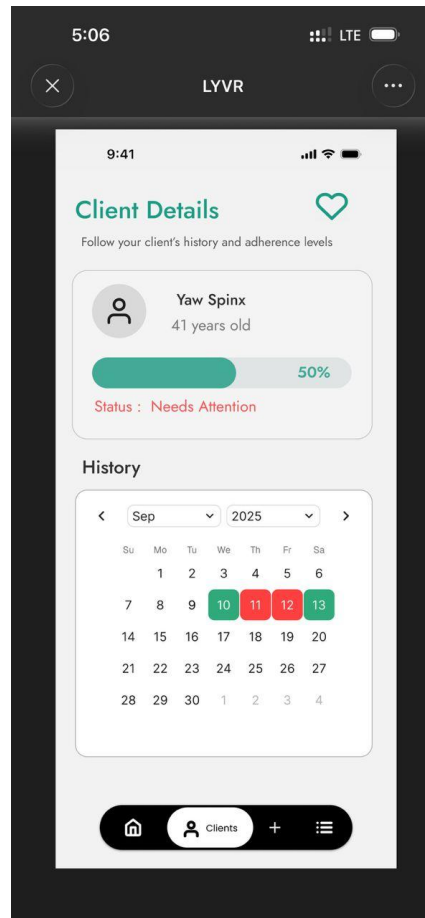


Figure 5: Caregiver Client Details — per-patient adherence bar, status flag, and calendar with taken (green) vs missed (red) days.

9.6 Caregiver Link Code (MorePage)

Each patient is assigned a unique numeric code displayed prominently in the Profile & Settings screen. Caregivers enter this code to establish a monitoring relationship. This is a deliberate, user-initiated pairing mechanism that respects patient privacy.

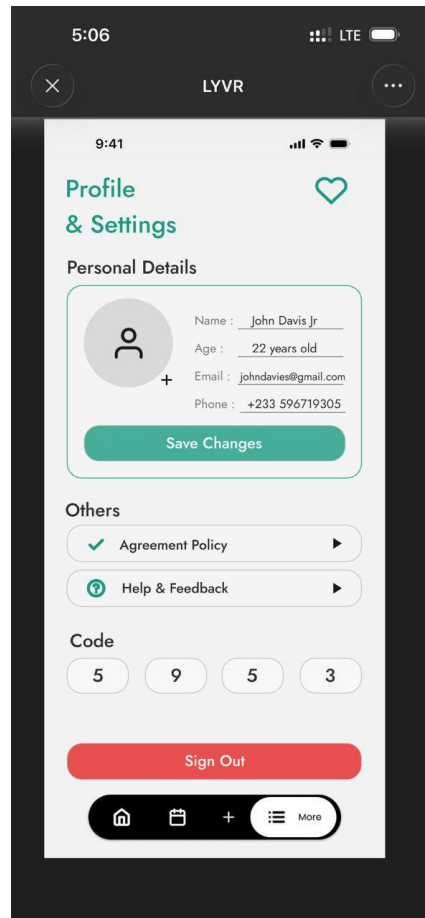


Figure 6: Profile & Settings (MorePage) — editable personal details and the 4-digit caregiver link code (5953) used for pairing.

10 Notes on Cloud / Backend Layer

The examination brief states cloud-based data persistence "if possible." The current MVP focuses on demonstrating a complete UI and architectural foundation. The recommended cloud backend is Firebase Firestore, which would handle:

- Patient authentication (Firebase Auth)
- Medication data synchronization across devices
- Caregiver-patient link management

- Push notification delivery (Firebase Cloud Messaging)
- Photo storage (Firebase Storage)

The local-first architecture of Ancora is designed to integrate cleanly with Firebase without structural changes to the existing screen and routing layer.

11. Technology Stack

Layer	Technology	Purpose
UI Framework	Flutter 3.x / Dart	Cross-platform widget rendering
Language	Dart (SDK ^3.11.4)	Application logic and state management
Design	Material Design 3	Consistent component library
Navigation	Flutter Named Routes	Decoupled screen routing
Local Storage (planned)	sqlite / Hive	Offline medication data persistence
Camera (planned)	image_picker / camera	Live photo capture for verification
Notifications (planned)	flutter_local_notifications	Scheduled medication reminders

Cloud (planned)	Firebase (Auth, Firestore, FCM, Storage)	Remote sync and caregiver connectivity
Platforms Supported	Android, iOS, Web, Linux, macOS, Windows	Cross-platform via Flutter

12. Conclusion

Ancora identifies a real, clinically significant problem in the healthcare sector and solves it with a thoughtfully designed, architecturally sound Flutter application.

The application implements a recognized mobile design pattern (Stateless/Stateful widget separation with centralized routing and theming), integrates device hardware (camera, notification system, clock), defines a complete data model with a local-first persistence strategy, and manages OS lifecycle events correctly through Flutter's widget lifecycle.

The responsive UI renders correctly across devices of all sizes using Flutter's adaptive layout system, and the dual-role architecture (patient and caregiver portals) demonstrates a mature understanding of mobile computing for healthcare environments.